TEMA 1. Clases y objetos

1. ¿Cuáles son las cuatro características básicas de la programación orientada a objetos? Describe brevemente cada una.

- La **abstracción** permite centrarse en lo esencial ocultando detalles de implementación, "olvidarse de los detalles". Permite manejar mejor temas complejos y facilita la modificación (el mantenimiento).

A partir de aquí, estas tres características "ayudan para olvidarse de los detalles":

- La **encapsulación** agrupa, "une", datos y operaciones relacionadas dentro de un artefacto, normalmente una clase, y controla su visibilidad.

- La **herencia** permite crear jerarquías entre clases que reutilizan y extienden el comportamiento de otras. Una subclase hereda atributos y métodos de su superclase y puede añadir o modificar funcionalidades, evitando duplicación de código y modelando relaciones “es-un”. ~~""En desuso""~~

- El **polimorfismo** se basa en **"misma función, distintas implementaciones en función del tipo"**. (Interfaces).

2. Cita cuatro lenguajes populares que permitan la programación orientada a objetos.

> - **Python** combina clases con tipado dinámico y soporte múltiple de paradigmas, lo que facilita prototipado rápido y legibilidad.

> - **Java** con comprobación estática de tipos. Es compliado, un proceso que detecta errores a la vez que traduce el código a una versión de bajo nivel ejecutable. Con Garbage Collector (recolección de basura) automática.

> - **C#** con comprobación estática de tipos. Es compliado, un proceso que detecta errores a la vez que traduce el código a una versión de bajo nivel ejecutable. Con Garbage Collector (recolección de basura) automática.

> - **C++** es un lenguaje compilado que añade orientación a objetos a C, permitiendo tanto programación procedural como orientada a objetos con control manual de memoria. No tiene Garbage Collector (recolección de basura) automática.

3. Los paradigmas anteriores a la POO, ¿Qué es la programación estructurada? y, todavía mejor, ¿Qué es la programación modular?

1. Con **ensamblador**, secuencias de instrucciones y saltos arbitrarios (`goto`), conocido como "código spaghetti", difícil mantenerlo a largo plazo y en proyectos grandes.

2. La **programación estructurada** añade secuencia, bifurcaciones (if, switch), iteraciones (while, for). Se elimita el `goto` .

3. La **programación modular** da un paso más, y se añaden conceptos como "librería", "paquete", "interfaz" que sirven para encapsular y reutilizar código.

❗ ❗ 4. ¿Qué tres elementos definen a un objeto en programación orientada a objetos?

Un **objeto** se define por:

1. **Identidad**, su instancia. Permite mantener referencias y comparar si dos variables apuntan al mismo objeto en memoria.
2. **Estado**, el valor de sus atributos o "campos".
3. **Comportamiento**, a través de sus métodos o "funciones". No deja de ser las cosas que "puede hacer" ese objeto.

5. ¿Qué es una clase? ¿Es lo mismo que un objeto? ¿Qué es una instancia? ¿Todos los lenguajes orientados a objetos manejan el concepto de clase?

❗ Una **clase** es un molde o "plantilla" que describe el estado y el comportamiento que compartirán los objetos creados a partir de ella: define atributos y métodos comunes. No es lo mismo que un objeto; la clase es el plano y el **objeto** es la entidad concreta creada siguiendo ese plano.

❗ Una **instancia** son objetos particulares construido a partir de una clase, con su propio estado en memoria. Varias instancias de la misma clase comparten el mismo conjunto de métodos pero mantienen valores de atributos independientes, lo que permite representar entidades similares con datos distintos.

No todos los lenguajes orientados a objetos exigen clases como mecanismo central. Java y C# se apoyan en clases, mientras que lenguajes como JavaScript usan un modelo basado en prototipos donde los objetos se enlazan directamente entre sí para heredar comportamiento, permitiendo orientación a objetos sin la declaración formal de clases tradicionales.

6. ¿Dónde se almacenan en memoria los objetos? ¿Es igual en todos los lenguajes? ¿Qué es la recolección de basura?

- En muchos lenguajes orientados a objetos los objetos se alojan en el **heap** (montón).
  - Presenta ventajas generales:
    - Memoria dinámica, se decide lo que se ocupa en tiempo de ejecución.
    - La vida de los objetos no depende de la vida de la función que los creó.
  - También presenta desventajas:
    - Hay que encargarse de liberar memoria no usada del heap, hay dos maneras:
      - Manual: Difícil y propenso a bugs (memory leaks).
      - Con recolector de basura.

- No es así en todos los lenguajes. Hay otros que permiten también el almacenamiento en el stack.

- La recolección de basura (Garbage Collector) es un proceso automático que identifica objetos inaccesibles y reclama su memoria. Presenta como "desventaja" su rendimiento, siempre busca si hay algo que liberar, y si no hay nada que liberar, malgasta recursos.

7. ¿Qué es un método? ¿Qué es la sobrecarga de métodos?

> - Un método son las funciones que un método puede hacer.

Un método es una operación asociada a una clase u objeto que define comportamiento: recibe parámetros, puede acceder al estado del objeto y devuelve un resultado o produce efectos. Actúa como la interfaz para interactuar con el objeto, encapsulando la lógica necesaria para consultar o modificar sus atributos de manera controlada.

> - La sobrecarga de métodos es la posibilidad de crear métodos dentro de una clase con el mismo nombre, pero cambiando el tipo y/o número de parámetros.

La sobrecarga de métodos consiste en definir varios métodos con el mismo nombre en una clase, diferenciados por su lista de parámetros (cantidad o tipos). El compilador selecciona la versión apropiada según los argumentos usados en la llamada, permitiendo ofrecer una misma operación con variantes adaptadas a distintos datos sin cambiar el nombre conceptual de la acción.

8. Ejemplo mínimo de clase en Java, que se llame Punto, con dos atributos, x e y, con un método que se llame `calculaDistanciaAOrigen`, que calcule la distancia a la posición 0,0. Por sencillez, los atributos deben tener visibilidad por defecto. Crea además un ejemplo de uso con una instancia y uso del método.

Una clase `Punto` puede declararse con atributos de visibilidad por defecto (sin modificador) y un método que compute la distancia al origen aplicando la fórmula $\sqrt{x^2 + y^2}$. El método devuelve un `double` y utiliza `Math.sqrt` para obtener la raíz cuadrada.

```java
class Punto {
        // Estado (atributos)
        int x;
        int y;

        // Constructor
        public Punto(int x, int y)  {
                this.x = x;
                this.y = y;
        }

        // Comportamiento (métodos)
        double calculaDistanciaAOrigen() {
                return Math.sqrt((x * x) + (y * y));
        }
}
```

Un uso básico consiste en crear una instancia con `new` , asignar valores a sus atributos y llamar al método para obtener la distancia. El resultado se puede imprimir por consola para verificar el cálculo.

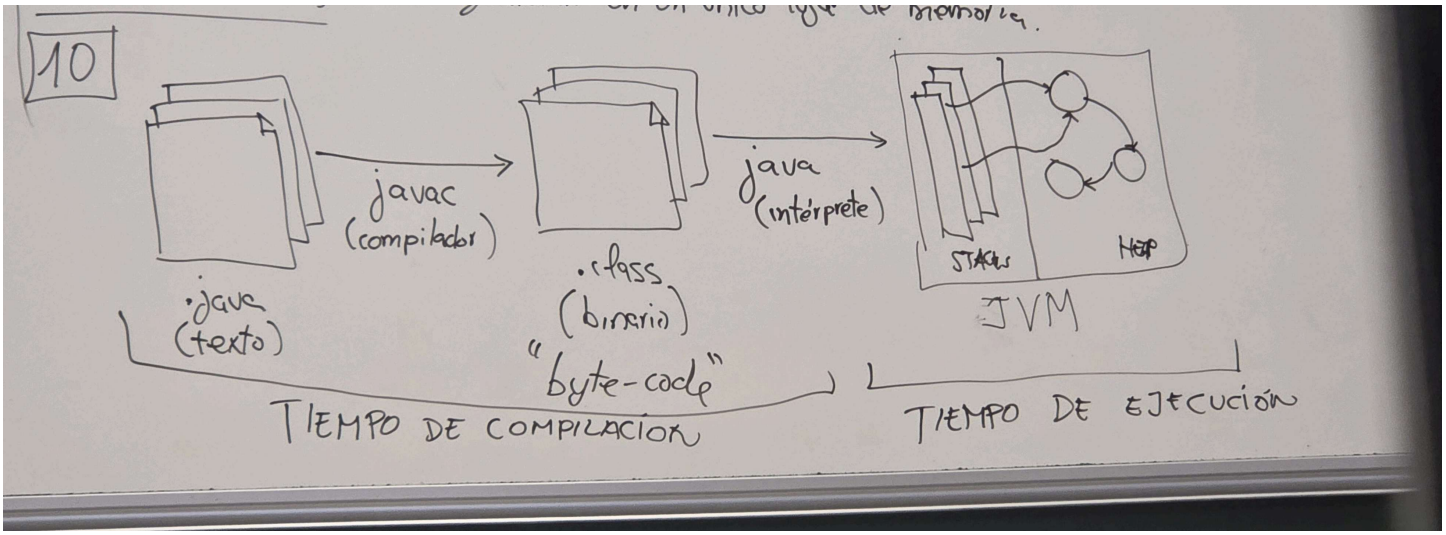
```java
public class EjemploUso {
        public static void main(String[] args) {
                Punto p = new Punto(3,4);
                double distancia = p.calculaDistanciaAOrigen();
                System.out.println("Distancia al origen: " + distancia); // Imprime 5.0
        }
}
```

9. ¿Cuál es el punto de entrada en un programa en Java? ¿Qué es `static` y para qué vale? ¿Sólo se emplea para ese método `main` ? ¿Para qué se combina con `final` ?

El punto de entrada de un programa Java es el método `main` con firma `public static void main(String[] args)` , que la JVM invoca al iniciar la aplicación. Este método debe ser `static` porque se ejecuta sin necesidad de crear una instancia de la clase contenedora, permitiendo que la JVM lo llame directamente.

- La palabra clave `static` permite usar un método o atributo sin necesidad de instancia de la clase (crear un objeto).
  - Se antepone el nombre de la clase al nombre del objeto.
  - Existen otros métodos además de `main` , como `Math.sqrt()` .
  - En estos métodos nunca existe `this` .
  - En atributos, solo se guardan en un único lugar de memoria.
- Combinado con `final` , que produce miembros inmutables, "constantes".
  `static final` se emplea para constantes (valores compartidos que no cambian, p.e: PI=3.1416) y puede aplicarse también a métodos para impedir que una subclase los sobreescriba.

10. Intenta ejecutar un poco de Java de forma básica, con los comandos `javac` y `java`.



¿Cómo podemos compilar el programa y ejecutarlo desde linea de comandos?

Para compilar un archivo fuente, se usa `javac NombreArchivo.java`, lo que genera un `.class` con bytecode (en binario). La ejecución se hace con `java NombreDeClase`, sin la extensión, y la JVM busca el método `main` correspondiente. Si hay varios archivos o paquetes, se incluye el classpath apropiado al compilar y ejecutar.

¿Java es compilado? ¿Qué es la máquina virtual?

Java se considera compilado e interpretado: el código fuente se compila a bytecode intermedio almacenado en ficheros `.class`. Ese bytecode lo interpreta o compila en tiempo de ejecución la Máquina Virtual de Java (JVM), que actúa como capa de abstracción sobre el sistema operativo y hardware, permitiendo la portabilidad del lema “escribe una vez, ejecuta en cualquier lugar”.

¿Qué es el *byte-code* y los ficheros `.class`?

El bytecode es un conjunto de instrucciones compactas e independientes de la plataforma que la JVM entiende. Los ficheros `.class` contienen ese bytecode; la JVM puede interpretarlo directamente o aplicar compilación JIT (just-in-time) para traducir partes a código nativo y mejorar el rendimiento mientras el programa se ejecuta.

11. En el código anterior de la clase `Punto`.

¿Qué es `new`?

Ocurren tres cosas al usar el operador `new`;

1. Reserva memoria en el heap y crea una nueva instancia de una clase (objeto)
2. Ejecuta un constructor con ese objeto como objeto actual (this).
3. Devuelve ese nuevo objeto, el NEW, **no** el constructor.

¿Qué es un constructor?

Un constructor tiene el mismo nombre que la clase y no devuelve valor. Puede recibir parámetros para establecer valores iniciales y garantizar que el objeto nace en un estado consistente. Si no se define, el lenguaje puede

proporcionar un constructor por defecto sin parámetros.

Pon un ejemplo de constructor en una clase `Empleado` que tenga DNI, nombre y apellidos.

```
class Empleado {
        private String dni;
        private String nombre;
        private String apellidos;

        Empleado(String dni, String nombre, String apellidos) {
                this.dni = dni;
                this.nombre = nombre;
                this.apellidos = apellidos;
        }
}
```

12. ¿Qué es la referencia `this` ? ¿Se llama igual en todos los lenguajes? Pon un ejemplo del uso de `this` en la clase `Punto` .

La referencia `this` identifica al objeto actual dentro de sus propios métodos, permitiendo acceder a sus atributos o invocar otros métodos sin ambigüedad. Resulta útil cuando nombres de parámetros o variables locales coinciden con los de los campos, o cuando se quieren encadenar llamadas al propio objeto.

No todos los lenguajes usan exactamente el mismo nombre, pero el concepto es equivalente: `this` en Java y C# o `self` en Python y Ruby, todos apuntan a la instancia receptora de la llamada.

```
class Punto {
        int x;
        int y;

        public Punto(int x, int y) {
                this.x = x; // distingue campos del objeto de parámetros
                this.y = y;
        }
}
```

13. Añade ahora otro nuevo método que se llame `distanciaA` , que reciba un `Punto` como parámetro y calcule la distancia entre `this` y el punto proporcionado.

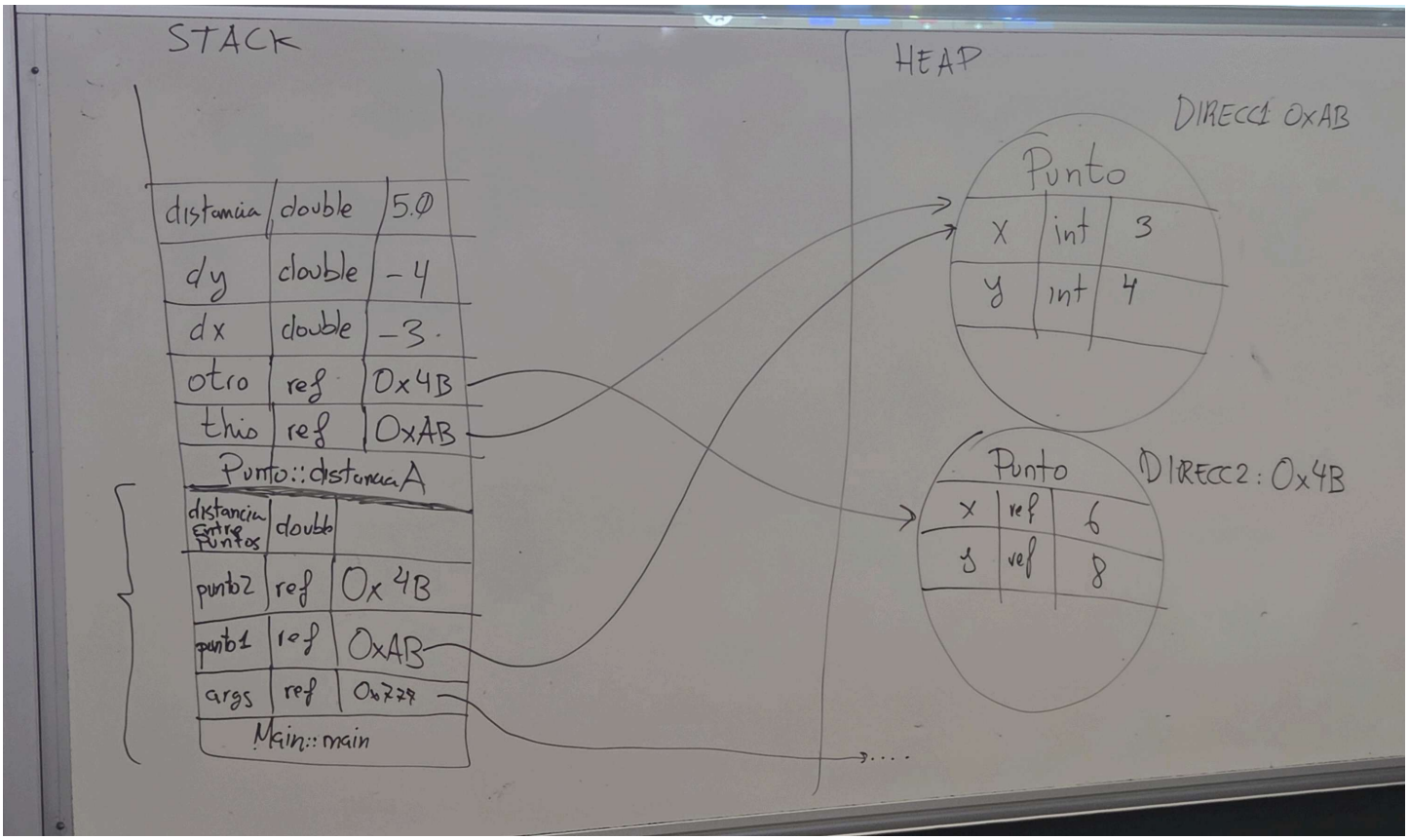
El método `distanciaA` puede calcular la distancia euclídea entre el punto actual y otro punto recibido como parámetro, restando coordenadas y aplicando la fórmula de distancia. Se apoya en `this` para referirse a los atributos del objeto que invoca el método.

```java
class Main {
        public static void main(String[] args) {
                Punto p1 = new Punto(3, 4);
                Punto p2 = new Punto(6, 8);
                double distancia = p1.distanciaA(p2); // Es p1.distanciaA(p2), no distanciaEntre(p1, p2)
                System.out.println("Distancia entre puntos: " + distancia);
        }
}

class Punto {
        int x;
        int y;

        double distanciaA(Punto otro) {
                int dx = this.x - otro.x;
                int dy = this.y - otro.y;
                return Math.sqrt((dx * dx) + (dy * dy));
        }
}
```

14. El paso del `Punto` como parámetro a un método,



¿es por copia o por referencia, es decir, si se cambia el valor de algún atributo del punto pasado como parámetro, dichos cambios afectan al objeto fuera del método?

En Java los parámetros se pasan por **valor**, pero en objetos, lo que se **copia es la referencia**.

Si un método recibe un `Punto` y modifica sus atributos mediante esa referencia, el objeto subyacente cambia y los

efectos son visibles fuera del método. Sin embargo, re-asignar la referencia dentro del método no altera la variable original del llamador.

¿Qué ocurre si en vez de un `Punto`, se recibiese un entero (`int`) y dicho entero se modificase dentro de la función?

Para tipos primitivos como `int`, el valor mismo se copia, de modo que modificar el parámetro dentro del método no afecta a la variable original. Se puede cambiar el contenido del parámetro local, pero el entero externo permanece igual tras la llamada, ya que no existe una referencia compartida al valor primitivo.

15. ¿Qué es el método `toString()` en Java? ¿Existe en otros lenguajes? Pon un ejemplo de `toString()` en la clase `Punto` en Java.

`toString()` es un método heredado de `Object` que devuelve una representación textual del objeto, pensada para depuración o salida legible. Sobrescribirlo permite describir el estado relevante en una cadena clara en lugar del formato por defecto que muestra la clase y un identificador hash.

Conceptos similares existen en otros lenguajes: `__str__` en Python, `toString` en JavaScript o `__toString` en PHP. Todos proporcionan una forma estándar de obtener una cadena representativa del objeto para mostrarla o registrarla.

```
class Punto {
        int x;
        int y;

        @Override
        public String toString() {
                return "Punto(" + x + ", " + y + ")";
        }
}
```

16. Reflexiona: ¿una clase es como un `struct` en C? ¿Qué le falta al `struct` para ser como una clase y las variables de ese tipo ser instancias?

Un `struct` en C agrupa datos bajo un mismo tipo, de forma similar a cómo una clase reúne atributos, pero carece de la asociación directa de comportamiento. No dispone de métodos, control de acceso, constructores ni mecanismos de encapsulación propios del modelo orientado a objetos.

Para asemejarse a una clase, al `struct` le faltarían funciones miembro con acceso implícito a su estado, inicialización controlada al crear objetos y visibilidad que oculte detalles internos. Las variables de un `struct` son simplemente bloques de datos sin identidad diferenciada más allá de su dirección, mientras que las instancias de una clase combinan estado, comportamiento y reglas de acceso en una unidad cohesiva.

17. Quitemos un poco de magia a todo esto: ¿Como se podría “emular”, con `struct` en C, la clase `Punto`, con su función para calcular la distancia al origen? ¿Qué ha pasado con `this`?

En C se puede emular una clase `Punto` definiendo un `struct` con los campos `x` e `y` y una función que reciba un puntero a ese `struct` para operar sobre sus datos. La función actúa como método libre y el puntero juega el papel de la referencia al objeto.

```
#include <math.h>

typedef struct {
        int x;
        int y;
} Punto;

double calculaDistanciaAOrigen(const Punto *p) {
        return sqrt((p->x * p->x) + (p->y * p->y));
}
```

Aquí `p` reemplaza al `this` implícito: al pasar el puntero se indica sobre qué instancia operar. La responsabilidad de asegurar inicialización y consistencia recae en quien crea y pasa el `struct`, ya que C no ofrece constructores ni control automático de acceso.